

consts.py

Project-wide configuration: paths, column groupings, hyperparameter defaults

This module holds every constant shared across the pipeline: filesystem paths, the raw Avazu column names grouped by role, and the default hyperparameters for the two learned feature-inducing components (the GBDT-leaf encoder and the Factorization Machine encoder, both defined in `feature_engineering.py`). Nothing in this file executes logic — it exists so that every other script imports its parameters from one place rather than repeating literals.

Path constants **CONST**

```
PROJECT_ROOT, DATA_DIR, RAW_DATA_DIR, PROCESSED_DATA_DIR, RESULTS_DIR, MODELS_DIR,
OUTPUTS_DIR, RAW_DATA_PATH
```

`PROJECT_ROOT` is resolved from `__file__`, so every other path is relative to wherever the repository is checked out rather than hardcoded. `DATA_DIR / RAW_DATA_DIR / PROCESSED_DATA_DIR` point at an in-repo `data/` tree (currently unused placeholders — the real dataset lives outside the repo).

`RESULTS_DIR / MODELS_DIR / OUTPUTS_DIR` are where `trainer.py` writes metrics JSON files and joblib model artifacts.

`RAW_DATA_PATH` is the one path that is *not* relative to the repo — it points at the real Avazu `train.gz` on disk (`/Users/gregorygoren/Documents/research/avazu-ctr-prediction/train.gz`), the full canonical Kaggle dataset (~40.4M rows, sorted by `hour`).

Column-name and run constants **CONST**

```
ID_COL, LABEL_COL, HOUR_COL, RANDOM_SEED, SAMPLE_N_ROWS, VAL_FRAC
```

`LABEL_COL = "click"` is the binary target column throughout the project. `HOUR_COL = "hour"` is Avazu's raw `YYMMDDHH` timestamp string, parsed by `feature_engineering.add_hour_features`.

`RANDOM_SEED` is the default seed threaded through every model/estimator for reproducibility (overridable via each script's `--seed` CLI flag). `SAMPLE_N_ROWS = 2,000,000` is the default number of leading rows `data_loader.load_sample` reads from the 40M-row raw file — a size chosen for fast iteration rather than using the full dataset. `VAL_FRAC = 0.2` is the fraction of the loaded sample held out as validation by `data_loader.time_based_split`.

Feature-column groupings **CONST**

```
USER_FEATURE_COLS, AD_FEATURE_COLS, CONTEXT_FEATURE_COLS, ALL_CAT_FEATURE_COLS,  
HOUR_DERIVED_COLS
```

Avazu has no explicit user or ad identifier, so these three lists group its raw columns by the role they play as identity proxies. `USER_FEATURE_COLS` (`device_id`, `device_ip`, `device_model`, `device_type`, `device_conn_type`) stand in for "who" is seeing the ad; `AD_FEATURE_COLS` (`site_id`, `site_domain`, `site_category`, `app_id`, `app_domain`, `app_category`) stand in for "what" is being advertised/where. Both are high-cardinality (some near-unique per row), which is why they get smoothed target-encoding in `feature_engineering.add_freq_agg_features` rather than plain one-hot.

`CONTEXT_FEATURE_COLS` (`C1`, `banner_pos`, `C14 - C21`) are low-cardinality anonymized context codes, safe to one-hot directly. `ALL_CAT_FEATURE_COLS` is the union of all three groups — every raw categorical column, used by the `baseline_ohc` feature set. `HOUR_DERIVED_COLS` names the two columns `feature_engineering.add_hour_features` derives from `hour` (`hour_of_day`, `day_of_week`) — named as a constant since different feature sets treat them as numeric or categorical.

GBDT_LEAF_ENCODER_PARAMS **CONST**

```
{"n_estimators": 100, "num_leaves": 31, "min_child_samples": 100, "feature_fraction":  
0.5, "bagging_fraction": 0.8, "bagging_freq": 1}
```

Default hyperparameters for the lightgbm GBDT that `feature_engineering.GBDTLeafEncoder` fits internally. These were tuned empirically on this project's time-based (non-i.i.d.) train/validation split: an initial 500-tree/64-leaf configuration measurably *overfit* (full-scale AUC regressed from 0.7385 to 0.7096), so the defaults were shrunk back to 100 trees/31 leaves and given explicit anti-overfitting regularization — `min_child_samples` raises the minimum leaf size well above lightgbm's default of 20, and `feature_fraction`/`bagging_fraction`+`bagging_freq` subsample columns/rows per tree so no single tree can lean too heavily on a memorization-prone near-unique column such as `device_id` or `device_ip`.

FM_ENCODER_PARAMS **CONST**

```
{"n_factors": 16, "n_epochs": 30, "batch_size": 4096, "learning_rate": 0.15, "l2_reg": 1e-5}
```

Default hyperparameters for `feature_engineering.FMEmbeddingEncoder`, the project's from-scratch Factorization Machine. `n_factors` is the latent embedding dimension k (kept modest since these become concatenated input features for a downstream linear model, not the final prediction). `n_epochs` is only an upper bound — training stops early once loss plateaus (see the `feature_engineering.pdf` documentation of `FMEmbeddingEncoder.fit`).

These values were raised from an initial 8-factor/5-epoch/ $lr=0.05$ pass that empirically undershot (the printed per-epoch logloss was still dropping at epoch 5). A sweep on a held-out sample found learning rate and epoch count to be the highest-leverage knobs, roughly doubling the encoder's own validation AUC before it was even concatenated with `freq_agg`'s features.